

SELF-REPAIRING MEMORY ARCHITECTURE — REUSABLE TEMPLATE

Condensed from “Build an Agent That Repairs Its Own Stale Memories” (@datasciencebrain)

Standalone reference — not project-specific. For reuse in any agent/memory build.

0. What this is

A condensed, implementation-agnostic template distilled from a 20-page build guide for an agent memory system that detects and repairs its own stale facts. The original guide is stack-specific (LangGraph, Groq, sqlite-vec, Python). This template keeps the **design decisions and reasoning**, strips the line-by-line code, so it can be applied later to any project — including a future Jarvis X memory layer — once you’re ready to merge it in with real feedback.

Use this as a checklist and reference architecture, not a spec to build verbatim.

1. The problem this solves

Every basic memory/RAG system stores facts and retrieves them by similarity. None of them handle a fact **going stale**.

Staleness has an asymmetry worth designing around:

- A memory that was **never true** is easy to catch — it contradicts something, retrieval scores it badly, a user corrects it.
- A memory that **was true** is the dangerous case. It embedded perfectly, retrieves with the highest score, and the model states it with total confidence — because until recently, nothing about it *was* wrong.

Two ways a stored fact becomes wrong: 1. **Somebody says something new** (a correction, easy to trigger on). 2. **Nobody says anything at all** — the fact outlived its shelf life, or the source document changed while no one was talking about it. Every purely event-driven design is blind to this second case, because there’s no event to fire on.

This template exists for case 2.

2. The core mental model

A memory is not a row. It is an interval with a belief attached.

Most stores treat a fact as a value you overwrite. This model treats it as two separate claims: - A claim that was true **for a stretch of time in the world** (valid time) - A claim the system **believed for a stretch of time** (transaction time)

These two stretches come apart constantly: you learn on June 10th that someone moved on June 3rd — valid time starts the 3rd, transaction time starts the 10th. Keeping both answers two different questions: - “*What’s true now?*” — which interval contains today - “*What did I believe in April?*” — which interval contains April

The four things that can happen to a memory

What happens	The interval	Still retrieved?	Example
Born	Opens, no end	Yes	“I live in Pune”
Replaced	Closes exactly where its successor opens	No — the successor is	“I moved to Bengaluru”
Ages	Stays open, belief weakens	Yes, flagged	Nobody mentioned the car in 8 months
Ends	Closes, nothing succeeds it	No	The dentist appointment happened

Nothing in this model is ever **deleted**. That's the one design decision everything else exists to protect — deletion destroys the ability to ever answer “what did I believe in March.”

3. Two paths into one writer

[illegible]

- **The arrival path** runs when the user says something: extract facts, compare against what's stored, decide.
- **The sweep path** runs on a timer when nobody has said anything: look for facts that have quietly gone bad — too old, source changed, window closed.

Both converge on **one writer module**. This is the single most reusable idea in the whole guide:

- One place to enforce every rule.
- One place to write the audit trail.
- Detection is cheap (SQL +

a file read) — a sweep over an unchanged store costs no model calls at all, which is what makes always-on monitoring affordable on a free tier.

4. Reusable configuration knobs

Every fact type needs a shelf life. Pick your own numbers, but the categories transfer:

Volatility class	Meaning	Example half-life
stable	Effectively never ages, but stays visible to the sweep so it's not a silent exemption	Years
slow	Changes on a multi-year horizon	Months
fast	Changes often ("my current project")	Weeks
scheduled	Doesn't decay — it <i>ends</i> , governed by an explicit end date, not by age	N/A

Two confidence thresholds, not one — **the asymmetry is the point**: destroying information should require more confidence than adding it.

Threshold	Suggested value	Rule
Auto-apply (retire a fact)	High (~0.80)	Being wrong here means losing information
Add/qualify a fact	Lower (~0.60)	Being wrong here just means a duplicate, not a loss
Always park for human review	—	Any contradiction on a <i>stable</i> fact (name, birthday, ID) — more often an extraction error than a real change

A “freshness floor” (e.g. 0.5) decides when a fact is flagged “may be out of date” at read time. Keep this the **same number** your sweep uses to decide “stale enough to flag” — two different thresholds for the same word will contradict each other in front of a user.

5. Reusable schema shape (adapt column names, keep the shape)

Truth table — one row per fact-version: - Identity: `user_id`, `topic` (closed vocabulary), `scope` (free-text qualifier — this is where ambiguity bugs live; getting `topic` vs `scope` wrong

is a common failure mode) - Content: text, value, volatility class, confidence - **Valid-time pair:** valid_from, valid_to - **Transaction-time pair:** recorded_at, expired_at - **Evidence-time field:** last_verified_at — distinct from recorded_at because aging should be measured from the last time you had *evidence* the fact still holds, not from when you first learned it. A fact learned two years ago and reconfirmed last week is fresh; a fact learned last week and never mentioned since is already starting to rot. - superseded_by (pointer to the replacing row), status (active / needs_verification / superseded / expired), source_kind + source_ref + source_hash (so source-document drift is detectable at all)

Sources table — store the **snapshot** a fact was extracted from, not just a pointer. Without the snapshot, you can never tell that a source document changed.

Repairs/audit table — append-only. Every mutation writes one row: what it was, what it became, the evidence, the confidence, who decided (agent / human / oracle), and a routing field (auto / parked / forced). This table earns its keep the first time someone asks “why did the agent think that.”

Open-ended sentinel instead of NULL — if your vector index or store can’t filter on IS NULL (common in vector extensions), use a deliberate far-future timestamp as “still open” instead of NULL, and use it consistently in every table so the two predicates never drift apart.

6. The sweep (the part most systems skip)

A background process, running on a clock, that finds rot **nobody reported:** - Facts past their volatility-class shelf life - Facts whose source document changed since last check - Facts whose valid-time window has closed with nothing marking it

Detection should be **pure lookups against your own metadata** — no model calls — so it’s cheap enough to run continuously. Only the *repair* step (deciding what to do about something the sweep found) should ever call a model, and only when confidence is genuinely ambiguous.

7. The human gate

Whenever the repair step isn’t confident enough to act alone, it **parks** the proposed change in an inbox instead of applying it — a lightweight review queue (a view over the audit table, not a second copy of the data) that a human clears asynchronously. This is the same pattern as a PR review: propose, don’t auto-merge, on anything above a defined risk bar.

8. A portable build order (bottom-up, each layer independently testable)

1. **Clock abstraction** — never call the system clock directly from business logic; inject a clock object everywhere. This is what lets you test “six months of decay” in one test run instead of waiting six months.
2. **Store** — schema + the bitemporal read/write helpers, before anything else touches it.
3. **Embed + recall** — retrieval that returns a fact’s age/confidence alongside its content, not just its text.
4. **Extract + classify + policy** — turn a message into structured facts, and decide what a new fact does to an old one (replace / age / coexist).
5. **Repair** — the one writer. Everything before this proposes; only this applies.
6. **Detect + propose + sweep** — the timer-driven path, built after arrival works, reusing the same repair endpoint.
7. **Human inbox** — parking + review UI, built last, once there’s something worth reviewing.

Each layer should be usable on its own before the next one starts — nothing should reference something that hasn’t been built yet.

9. Checklist for applying this template to a new project

- ☐ Does every fact type have an assigned volatility class and shelf life?
 - ☐ Are valid-time and transaction-time kept as separate column pairs, not one timestamp?
 - ☐ Is there a clock abstraction, or does business logic call the system clock directly?
 - ☐ Is “delete” actually used anywhere in the live code path? (It shouldn’t be.)
 - ☐ Do retrieval and the sweep use the *same* staleness threshold?
 - ☐ Is there a single writer module that both paths (arrival + sweep) call?
 - ☐ Is detection free of model calls? Is repair the only step that spends money?
 - ☐ Is there an append-only audit row for every mutation, including the “before” state?
 - ☐ Is there a confidence-gated human inbox for anything that isn’t a clear auto-apply?
 - ☐ Are two thresholds used — one for adding, one (higher) for retiring — rather than one shared number?
-

Source: a 20-page/22-section public build guide (@datasciencebrain), condensed to its transferable architecture. Original stack (LangGraph + Groq + sqlite-vec + sentence-transformers, Python/Windows) omitted here since this template is meant to outlive any one implementation.